Studio Nx

S1-05 오디오 엔진 기초 — 함수 설명 합본 (JS 104~114)

2026-10-14

목차

1. ai-notes/javascript/S1-05 - 오디오 엔진 기초/104_timing_ticksToSeconds.md
2. ai-notes/javascript/S1-05 - 오디오 엔진 기초/105_timing_ticksToSecondsWithChanges.md
3. ai-notes/javascript/S1-05 - 오디오 엔진 기초/106_timing_secondsToTicks.md
4. ai-notes/javascript/S1-05 - 오디오 엔진 기초/107_scheduler_planSchedule.md
5. ai-notes/javascript/S1-05 - 오디오 엔진 기초/108_scheduler_Transport.md
6. ai-notes/javascript/S1-05 - 오디오 엔진 기초/109_scheduler_Transport_position.md
7. ai-notes/javascript/S1-05 - 오디오 엔진 기초/110_scheduler_setTempo.md
8. ai-notes/javascript/S1-05 - 오디오 엔진 기초/111_metronome_metronomeClick.md
9. ai-notes/javascript/S1-05 - 오디오 엔진 기초/112_metronome_Metronome.md
10. ai-notes/javascript/S1-05 - 오디오 엔진 기초/113_scheduler_loop.md
11. ai-notes/javascript/S1-05 - 오디오 엔진 기초/114_engine_AudioEngine.md

JS 104 — `ticksToSeconds()` (tick를 초로 바꾸기)

항목	내용
파일	`studionx/frontend/static/js/audio/timing.js` (새 파일, 소리와 화면을 쓰지 않는 계산 전용 모듈)
작업	S1-05 오디오 엔진 기초: 첫 번째 함수 (FR-205 시간 단위 tick, PPQ 960)
작성	Claude (AI), 2026-10-13, 사용자 검토
브랜치	`feature/phase1/s1-05-audio-engine` (`develop`에서 만듦)
다음	곡 중간 템포 변경을 반영한 변환(`tempo_changes`), 스케줄러, 메트로놈, 루프 순서로 한 함수씩 진행합니다

1. 하는 일

시간을 **tick**(1박 = 960 tick)으로 관리하고, 소리를 낼 때만 **초**로 바꿉니다. 이 함수는 템포(BPM)가 하나일 때의 변환입니다.

> 초 = tick × 60 ÷ (BPM × 960)

입력	결과
960 tick(1박), 120 BPM	0.5초
3840 tick(4/4 한 마디), 120 BPM	2초
960 tick, 60 BPM	1초
-960 tick (카운트인처럼 0보다 앞), 120 BPM	-0.5초
BPM이 0 이하, 숫자가 아님, tick이 NaN이나 Infinity	오류(`RangeError`)

2. 코드

```
export const PPQ = 960;

export function ticksToSeconds(tick, bpm) {
  if (typeof tick !== 'number' || !isFinite(tick)) throw new RangeError('tick 은 유한한 숫자여야 합니다: ' + tick);
  if (typeof bpm !== 'number' || !isFinite(bpm) || bpm <= 0) throw new RangeError('bpm 은 0 보다 큰 숫자여야 합니다: ' + bpm);
  return (tick * 60) / (bpm * PPQ);
}
```

최신 문법으로 쓰면 (공부용)

```
export const ticksToSeconds = (tick, bpm) => {
  if (!Number.isFinite(tick)) throw new RangeError(`tick 은 유한한 숫자여야 합니다: ${tick}`);
  if (!Number.isFinite(bpm) || bpm <= 0) throw new RangeError(`bpm 은 0 보다 큰 숫자여야 합니다: ${bpm}`);
  return (tick * 60) / (bpm * PPQ);
};
```

최신 문법	실제 코드	뜻
`Number.isFinite(x)`	`typeof x !== 'number' || !isFinite(x)`	숫자이면서 NaN, Infinity가 아닌지 확인합니다. `Number.isFinite`는 문자열을 자동 변환하지 않아서 한 번에 됩니다
템플릿 문자열 `` `…${x}` ``	`'…' + x`	글자와 값을 잇습니다
화살표 함수	`function` 선언	같은 동작입니다

3. 왜 이렇게

선택	이유
시간은 tick, 소리를 낼 때만 초	초로 저장하면 BPM을 바꿀 때 모든 클립의 위치를 다시 계산해야 합니다. tick이면 템포가 바뀌어도 위치가 변하지 않습니다 (DB와 같은 단위)
`audio/` 폴더에 DOM과 `AudioContext`를 쓰지 않는 파일로 분리	계산만 따로 시험할 수 있습니다. 나중에 오프라인 렌더(WAV 출력, S1-11)에서도 같은 계산을 그대로 씁니다
잘못된 값은 오류를 던짐	`NaN`이나 `Infinity`를 조용히 돌려주면 스케줄러가 엉뚱한 시각에 소리를 냅니다. 바로 알리는 쪽이 원인을 찾기 쉽습니다
BPM의 범위(20~300)는 여기서 검사하지 않음	범위는 입력 화면과 저장(FR-202)이 맡고, 이 함수는 계산이 성립하는 조건(0보다 큰 유한한 수)만 봅니다
tick이 음수여도 계산	카운트인(FR-203)처럼 0보다 앞의 시간을 쓸 수 있어야 합니다

4. 시험 (Node에서 직접 실행, 16개 통과)

확인	결과
120 BPM 1박 0.5초, 60 BPM 1박 1초, 0 tick 0초, 한 마디 2초	✓
소수 BPM(93.5), 경계 BPM(20은 3초, 300은 0.2초)	✓
음수 tick(-960은 -0.5초), `PPQ`가 960	✓
BPM이 0, 음수, 문자열, NaN이면 오류	✓
tick이 NaN, Infinity, 없음이면 오류	✓

JS 105 — `ticksToSecondsWithChanges()` (곡 중간 템포 변경을 반영한 tick→초)

항목	내용
파일	`studionx/frontend/static/js/audio/timing.js` (함수 추가, 소리와 화면을 쓰지 않는 계산 전용 모듈)
작업	S1-05 오디오 엔진 기초: 두 번째 함수 (FR-202a 곡 중간 템포 변경, FR-205 시간 단위 tick, PPQ 960)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-05-audio-engine`
다음	스케줄러(tick 을 보고 다음 소리 시각을 예약), 메트로놈, 루프 순서로 한 함수씩 진행합니다

1. 하는 일

곡 중간에 BPM 이 바뀌는 경우를 반영해 tick을 초로 바꿉니다. `projects.bpm` 은 곡의 시작(tick 0) 템포이고, `tempo_changes` 표의 변경점들을 tick 순서대로 지나가며 구간마다 초를 더합니다.

입력	결과
120 BPM, 변경 없음, 960 tick(1박)	0.5초 (앞 함수 104와 같음)
120→60 STEP 이 3840 tick 뒤, 그 위치	2초 (앞 구간 2초)
같은 곡, 그 위치에서 960 tick 뒤	3초 (60 BPM 으로 1박 = 1초 더함)
120→240 LINEAR 이 3840 tick 에 걸쳐, 그 위치	약 1.386초 (로그 적분)
tick 이 음수 (카운트인)	시작 템포 하나로 계산

`STEP` 은 변경점 위치에서 바로 바뀌고, `LINEAR` 는 바로 앞 변경점 위치부터 이 변경점까지 서서히 바뀝니다 (`DB.md` `tempo_changes.ramp`).

2. 코드

```
export function ticksToSecondsWithChanges(tick, baseBpm, changes) {
  if (typeof tick !== 'number' || !isFinite(tick)) throw new RangeError('tick 은 유한한 숫자여야 합니다: ' + tick);
  if (typeof baseBpm !== 'number' || !isFinite(baseBpm) || baseBpm <= 0) throw new RangeError('baseBpm 은 0 보다 큰 숫자여야 합니다: ' + baseBpm);

  const list = [];
  if (changes != null) {
    if (!Array.isArray(changes)) throw new RangeError('changes 는 배열이어야 합니다: ' + changes);
    for (let i = 0; i < changes.length; i++) {
      const c = changes[i];
      if (c == null) continue;
      if (typeof c.tick !== 'number' || !isFinite(c.tick) || c.tick <= 0) throw new RangeError('변경점 tick 은 0 보다 큰 유한한 숫자여야 합니다: ' + c.tick);
      if (typeof c.bpm !== 'number' || !isFinite(c.bpm) || c.bpm <= 0) throw new RangeError('변경점 bpm 은 0 보다 큰 숫자여야 합니다: ' + c.bpm);
      list.push({ tick: c.tick, bpm: c.bpm, ramp: c.ramp === 'LINEAR' ? 'LINEAR' : 'STEP' });
    }
```

```
    list.sort(function (a, b) { return a.tick - b.tick; });
  }

  if (tick <= 0) return (tick * 60) / (baseBpm * PPQ);

  let seconds = 0;
  let prevTick = 0;
  let bpm = baseBpm;
  for (let i = 0; i < list.length; i++) {
    const c = list[i];
    if (c.tick <= prevTick) continue;
    const segEnd = tick < c.tick ? tick : c.tick;
    if (c.ramp === 'LINEAR' && c.bpm !== bpm) {
      const span = c.tick - prevTick;
      const slope = (c.bpm - bpm) / span;
      const bpmAtEnd = bpm + slope * (segEnd - prevTick);
      seconds += (60 / PPQ) * (1 / slope) * Math.log(bpmAtEnd / bpm);
    } else {
      seconds += ((segEnd - prevTick) * 60) / (bpm * PPQ);
    }
    prevTick = c.tick;
    bpm = c.bpm;
    if (tick <= c.tick) return seconds;
  }
  seconds += ((tick - prevTick) * 60) / (bpm * PPQ);
  return seconds;
}
```

최신 문법으로 쓰면 (공부용)

```
export const ticksToSecondsWithChanges = (tick, baseBpm, changes) => {
  const list = (changes ?? [])
    .filter((c) => c != null)
    .map((c) => ({ tick: c.tick, bpm: c.bpm, ramp: c.ramp === 'LINEAR' ? 'LINEAR' : 'STEP' }))
    .sort((a, b) => a.tick - b.tick);
  // …나머지 계산은 같음
};
```

최신 문법	실제 코드	뜻
`changes ?? []`	`changes != null ? … : []`	값이 없으면 빈 배열을 씁니다. 실제 코드는 `if (changes != null)` 로 나누어 검사합니다
`.filter().map().sort()`	`for` 반복문	배열을 걸러 내고 바꾸고 정렬합니다
화살표 함수	`function` 선언	같은 동작입니다

3. 왜 이렇게

선택	이유
앞 함수 `ticksToSeconds` 를 고치지 않고 새 함수로 만듦	템포가 하나일 때는 104가 더 단순하고 빠릅니다. 스케줄러는 "변경점이 없으면 104, 있으면 105" 로 고르면 됩니다
구간을 차례로 더해 나감	변경점은 보통 몇 개 안 되므로 매번 처음부터 더해도 충분합니다. 나중에 자주 부르게 되면 누적 시간을 미리 계산하는 함수로 바꿀 수 있습니다

선택	이유
`LINEAR` 는 로그로 계산	BPM 이 tick 에 대해 선형으로 변할 때 초는 `1/BPM` 을 적분한 값이라 `ln` 이 나옵니다. 구간을 잘게 나누어 더하는 근사 대신 정확한 값을 씁니다
`LINEAR` 인데 시작·끝 BPM 이 같으면 상수 처리	`ln(1)=0` 이고 `slope=0` 이라 0으로 나누게 됩니다. 이 경우는 상수 구간과 같으므로 나누지 않고 그대로 더합니다
`ramp` 값이 `LINEAR` 가 아니면 `STEP` 으로	`DB.md` 의 ENUM 은 `STEP`·`LINEAR` 둘뿐입니다. 모르는 값이 와도 소리를 내는 계산이 멈추지 않게 기본값을 둡니다
정렬은 함수 안에서	호출하는 쪽이 정렬을 잊어도 결과가 틀리지 않게 합니다. 같은 tick 이 겹치면 뒤 값이 이깁니다
음수 tick 은 시작 템포로	카운트인(FR-203)은 0보다 앞이라 템포 변경이 없습니다. 104와 같은 식을 씁니다
`changes` 를 그대로 받음	데이터베이스에서 받은 `tempo_changes` 행을 그대로 넣을 수 있게 했습니다. 필요한 칸(`tick`·`bpm`·`ramp`)만 읽습니다

4. 시험 (Node에서 직접 실행, 23개 통과)

확인	결과
변경 없음·`null` 일 때 104와 같은 값(0.5초, 한 마디 2초)	✓
STEP 변경점 위치·뒤·앞 계산 (2초, 3초, 1초)	✓
STEP 두 번 (120→60→120) 구간 더하기	✓
LINEAR 전체 구간이 로그 적분 값과 일치	✓
LINEAR 구간 절반 지점(BPM 180) 값	✓
LINEAR 인데 BPM 이 같으면 상수(2초)	✓
정렬하지 않은 입력도 정렬되어 같은 결과	✓
음수 tick(−960 = −0.5초)	✓
tick · baseBpm · changes · 변경점 tick·bpm 오류 시 `RangeError`	✓
알 수 없는 `ramp` 값은 STEP 처리	✓

JS 106 — `secondsToTicks()` (초를 tick으로 되돌리기)

항목	내용
파일	`studionx/frontend/static/js/audio/timing.js` (함수 추가, 소리와 화면을 쓰지 않는 계산 전용 모듈)
작업	S1-05 오디오 엔진 기초: 세 번째 함수 (FR-202a 곡 중간 템포 변경의 역변환, FR-205 시간 단위 tick, PPQ 960)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-05-audio-engine`
다음	스케줄러(재생 중 위치를 tick 으로 잡고 다음 소리 시각을 예약), 메트로놈, 루프 순서로 한 함수씩 진행합니다

1. 하는 일

`ticksToSecondsWithChanges` (105) 의 역함수입니다. 초(재생 위치)를 tick으로 되돌립니다. 재생 중인 위치를 화면에 표시하거나, 루프 끝을 tick으로 잡을 때 씁니다.

입력	결과
0.5초, 120 BPM, 변경 없음	960 tick (1박)
2초, 120 BPM, 변경 없음	3840 tick (한 마디)
120→60 STEP 곡에서 3초	3840 + 960 = 4800 tick
120→240 LINEAR 곡에서 3840 tick의 초	3840 tick (왕복 확인)
−0.5초 (카운트인)	−960 tick

`ticksToSecondsWithChanges(secondsToTicks(초))` 가 원래 초로, `secondsToTicks(ticksToSecondsWithChanges(tick))` 가 원래 tick으로 돌아옵니다(왕복 시험).

2. 코드

```
export function secondsToTicks(seconds, baseBpm, changes) {
  if (typeof seconds !== 'number' || !isFinite(seconds)) throw new RangeError('seconds 는 유한한 숫자여야 합니다: ' + seconds);
  if (typeof baseBpm !== 'number' || !isFinite(baseBpm) || baseBpm <= 0) throw new RangeError('baseBpm 은 0 보다 큰 숫자여야 합니다: ' + baseBpm);

  const list = [];
  if (changes != null) {
    if (!Array.isArray(changes)) throw new RangeError('changes 는 배열이어야 합니다: ' + changes);
    for (let i = 0; i < changes.length; i++) {
      const c = changes[i];
      if (c == null) continue;
      if (typeof c.tick !== 'number' || !isFinite(c.tick) || c.tick <= 0) throw new RangeError('변경점 tick 은 0 보다 큰 유한한 숫자여야 합니다: ' + c.tick);
      if (typeof c.bpm !== 'number' || !isFinite(c.bpm) || c.bpm <= 0) throw new RangeError('변경점 bpm 은 0 보다 큰 숫자여야 합니다: ' + c.bpm);
      list.push({ tick: c.tick, bpm: c.bpm, ramp: c.ramp === 'LINEAR' ? 'LINEAR' : 'STEP' });
    }
    list.sort(function (a, b) { return a.tick - b.tick; });
  }
```

```
  if (seconds <= 0) return (seconds * baseBpm * PPQ) / 60;

  let rest = seconds;
  let prevTick = 0;
  let bpm = baseBpm;
  for (let i = 0; i < list.length; i++) {
    const c = list[i];
    if (c.tick <= prevTick) continue;
    const span = c.tick - prevTick;
    const isLinear = c.ramp === 'LINEAR' && c.bpm !== bpm;
    const slope = isLinear ? (c.bpm - bpm) / span : 0;
    const segSeconds = isLinear
      ? (60 / PPQ) * (1 / slope) * Math.log(c.bpm / bpm)
      : (span * 60) / (bpm * PPQ);

    if (rest >= segSeconds) {
      rest -= segSeconds;
      prevTick = c.tick;
      bpm = c.bpm;
      continue;
    }
    if (isLinear) {
      const bpmAt = bpm * Math.exp((rest * PPQ * slope) / 60);
      return prevTick + (bpmAt - bpm) / slope;
    }
    return prevTick + (rest * bpm * PPQ) / 60;
  }
  return prevTick + (rest * bpm * PPQ) / 60;
}
```

최신 문법으로 쓰면 (공부용)

```
export const secondsToTicks = (seconds, baseBpm, changes) => {
  let rest = seconds;
  let prevTick = 0;
  let bpm = baseBpm;
  for (const c of (changes ?? [])) {
    // …구간 계산은 같음
  }
  return prevTick + (rest * bpm * PPQ) / 60;
};
```

최신 문법	실제 코드	뜻
`for (const c of arr)`	`for (let i = 0; …)`	배열의 값을 차례로 씁니다
`changes ?? []`	`if (changes != null) …`	값이 없으면 빈 배열로 봅니다
`Math.exp`	(같음)	실제 코드도 `Math.exp` 를 씁니다. `ln` 의 역으로 지수(e의 거듭제곱)를 구합니다

3. 왜 이렇게

선택	이유
구간을 차례로 지나가며 "남은 초"를 줄여 나감	105와 같은 방식이라 두 함수의 결과가 서로 맞습니다. 변경점이 몇 개뿐이라 충분히 빠릅니다
LINEAR 은 지수(`Math.exp`)로 풂	105에서 초 = 60/PPQ × (1/slope) × ln(bpm(t)/bpmStart) 였으므로, 이를 bpm(t) 에 대해 풀면 지수가 됩니다. `bpm(t)` 을 구한 뒤 `t = prevTick + (bpm(t)-bpmStart)/slope` 로 위치를 냅니다
`rest >= segSeconds` 로 구간 경계 처리	남은 초가 구간 길이와 정확히 같으면 다음 구간으로 넘겨 그 변경점 위치를 돌려줍니다. 왕복 시 경계에서 값이 어긋나지 않습니다
초가 음수면 시작 템포로	카운트인(FR-203)은 0보다 앞이라 템포 변경이 없습니다. 105의 음수 처리와 역이 맞습니다
소수점 tick 을 그대로 돌려줌	반올림(마디·박 표시)은 부르는 쪽이 정합니다. 여기서 미리 반올림하면 왕복 값이 틀어집니다
오류 검사는 105와 같게	같은 입력 규칙(tick 은 0보다 큼, bpm 은 0보다 큼)을 씁니다

4. 시험 (Node에서 직접 실행, 37개 통과 · 105의 23개도 다시 통과)

확인	결과
기본 값(0.5초=960, 2초=3840, 0초=0, −0.5초=−960)	✓
왕복: tick→초→tick 18가지(변경 없음, STEP 1·2개, LINEAR 오르막·내리막, 섞임)	✓
왕복: 초→tick→초 6가지	✓
변경점 경계에서 정확히 그 tick	✓
정렬하지 않은 입력	✓
seconds · baseBpm · changes · 변경점 값 오류 시 `RangeError`	✓

JS 107 — `planSchedule()` (스케줄러가 예약할 소리 목록 계산)

항목	내용
파일	`studionx/frontend/static/js/audio/scheduler.js` (새 파일, 소리와 시계를 쓰지 않는 스케줄러 계산 부분)
작업	S1-05 오디오 엔진 기초: 네 번째 함수 (FR-201 재생, FR-205 tick/PPQ 960, FR-202a 템포 변경)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-05-audio-engine`
다음	이 계산을 쓰는 실제 스케줄러 루프(`Transport`: AudioContext + setInterval lookahead), 그다음 메트로놈, 루프

1. 하는 일

브라우저에서 소리를 정확한 시각에 내려면 **lookahead 방식**을 씁니다. 짧은 타이머(약 25ms)가 자주 돌면서 "앞으로 0.1초 안에 울릴 소리"를 미리 `AudioContext` 시각으로 예약합니다. 이 함수는 그중에서 "지금 예약할 소리 목록"을 계산하는 **순수 계산** 부분입니다(소리·시계 없음 → Node로 시험 가능).

커서(tick, time)는 "아직 예약하지 않은 다음 소리"를 가리킵니다. `time`이 `untilTime`보다 작은 동안 커서를 `stepTicks`씩 옮기며 목록에 담고, 이어 쓸 커서(`nextTick`·`nextTime`)를 돌려줍니다.

입력	결과
커서 (0 tick, 0초), 1/16=240 tick, 120 BPM, until 0.1초	0.125초 간격으로 tick 0, 240, ..., 720 (8개)
커서 (3600 tick) 에서 변경점 3840(120→60)을 넘는 창	3840 뒤 단계의 간격이 0.125초 → 0.25초로 바뀜
untilTime ≤ fromTime	빈 목록, 커서 그대로

2. 코드

```
import { ticksToSecondsWithChanges } from './timing.js';

export const MAX_EVENTS_PER_PLAN = 100000;

export function planSchedule(fromTick, fromTime, untilTime, stepTicks, baseBpm, changes) {
  if (typeof fromTick !== 'number' || !isFinite(fromTick)) throw new RangeError('fromTick 은 유한한 숫자여야 합니다: ' + fromTick);
  if (typeof fromTime !== 'number' || !isFinite(fromTime)) throw new RangeError('fromTime 은 유한한 숫자여야 합니다: ' + fromTime);
  if (typeof untilTime !== 'number' || !isFinite(untilTime)) throw new RangeError('untilTime 은 유한한 숫자여야 합니다: ' + untilTime);
  if (typeof stepTicks !== 'number' || !isFinite(stepTicks) || stepTicks <= 0) throw new RangeError('stepTicks 는 0 보다 큰 숫자여야 합니다: ' + stepTicks);
  if (typeof baseBpm !== 'number' || !isFinite(baseBpm) || baseBpm <= 0) throw new RangeError('baseBpm 은 0 보다 큰 숫자여야 합니다: ' + baseBpm);

  const events = [];
  let tick = fromTick;
  let time = fromTime;
  while (time < untilTime) {
    if (events.length >= MAX_EVENTS_PER_PLAN) throw new RangeError('한 번에 예약할 단계가 너무 많습니다 ('
```

```
+ MAX_EVENTS_PER_PLAN + '개 초과)');
    events.push({ tick: tick, time: time });
    const nextTick = tick + stepTicks;
    const delta = ticksToSecondsWithChanges(nextTick, baseBpm, changes) - ticksToSecondsWithChanges(tic
k, baseBpm, changes);
    tick = nextTick;
    time += delta;
  }
  return { events: events, nextTick: tick, nextTime: time };
}
```

최신 문법으로 쓰면 (공부용)

```
export const planSchedule = (fromTick, fromTime, untilTime, stepTicks, baseBpm, changes) => {
  const events = [];
  let tick = fromTick;
  let time = fromTime;
  while (time < untilTime) {
    events.push({ tick, time });
    const nextTick = tick + stepTicks;
    time += ticksToSecondsWithChanges(nextTick, baseBpm, changes) - ticksToSecondsWithChanges(tick, base
Bpm, changes);
    tick = nextTick;
  }
  return { events, nextTick: tick, nextTime: time };
};
```

최신 문법	실제 코드	뜻
`{ tick, time }` (속기)	`{ tick: tick, time: time }`	이름이 같은 변수를 그대로 담습니다
`while …`	(같음)	실제 코드도 `while` 을 씁니다

3. 왜 이렇게

선택	이유
소리·시계와 분리한 순수 함수	소리 없이 Node로 계산을 시험할 수 있습니다. 소리를 내는 부분(`Transport`)은 이 결과를 받아 `AudioContext` 노드만 만듭니다
`time` 은 "절대 시각"으로 받음	커서 시각은 `AudioContext.currentTime + 여유` 같은 절대 시각입니다. 구간마다 더하는 값(차이)만 쓰므로 절대·상대 어느 쪽이어도 맞습니다
단계 초를 105의 차이로 구함	`ticksToSecondsWithChanges(t+step) - ticksToSecondsWithChanges(t)` 로 하면 한 단계가 템포 변경점을 걸쳐도 정확합니다(예: 3840을 넘는 단계)
커서를 돌려줌	스케줄러가 다음 타이머 때 이어서 쓰도록 합니다. 매번 처음부터 다시 계산하지 않습니다
`MAX_EVENTS_PER_PLAN` 안전 한계	`untilTime` 이 너무 멀거나 `stepTicks` 가 너무 작으면(잘못된 호출) 탭이 멈출 수 있습니다. 그때 오류로 알립니다. 정상 호출(0.1초 창)은 몇 개 안 됩니다
`baseBpm` 은 여기서도 검사	`untilTime ≤ fromTime` 이면 반복이 한 번도 안 돌아 105가 검사할 기회가 없습니다. 그래서 여기서도 검사합니다

4. 시험 (Node에서 직접 실행, 26개 통과)

확인	결과
120 BPM 1/16 창(0.1초): tick 0,240,… 간격, 시간 간격 0.125초	✓
모든 이벤트가 untilTime 미만, 커서의 time 은 untilTime 이상	✓
이벤트 time 이 `ticksToSecondsWithChanges` 직접 계산과 일치	✓
untilTime ≤ fromTime 이면 빈 목록, 커서 그대로	✓
연속 호출 시뮬레이션: 커서가 이어져 tick이 240씩 끊김 없이 증가, 시간 단조 증가	✓
변경점(STEP)을 걸친 창: 3840 뒤 간격이 0.125→0.25초	✓
LINEAR 곡에서도 tick이 240씩	✓
안전 한계 초과 시 `RangeError`, 값 오류 7종	✓

JS 108 — `Transport` (재생 스케줄러 루프)

항목	내용
파일	`studionx/frontend/static/js/audio/scheduler.js` (같은 파일에 클래스 추가)
작업	S1-05 오디오 엔진 기초: 다섯 번째 함수 (FR-201 재생/정지, FR-205 tick/PPQ 960)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-05-audio-engine`
다음	pause/resume와 재생 중 위치 이동(seek), 재생 중 템포 변경, 그다음 메트로놈, 루프

1. 하는 일

107의 `planSchedule`(순수 계산)을 실제 소리 시각에 얹는 루프입니다. `clock`(브라우저에서는 `AudioContext`)에서 지금 시각을 읽어 "앞으로 lookahead(0.1초) 안에 울릴 소리"를 `onStep(tick, time)`으로 넘기고, 커서를 옮깁니다. 타이머(약 25ms)가 이 `pump()`를 반복해서 부릅니다.

값	기본	뜻
`lookahead`	0.1초	앞으로 이만큼 미리 예약합니다
`intervalMs`	25	타이머가 도는 간격입니다
`startDelay`	0.05초	시작할 때 첫 소리를 조금 뒤로 미룹니다
`stepTicks`	240 (1/16)	한 단계에 나아갈 tick 입니다 (PPQ/4)

`clock`(필수)과 `setTimer·clearTimer`(기본 전역 `setInterval·clearInterval`)를 주입받게 만들어, Node에서 가짜 시계와 가짜 타이머로 소리 없이 시험합니다.

2. 코드

```
export class Transport {
  constructor(options) {
    options = options || {};
    if (options.clock == null || typeof options.clock.now !== 'function') {
      throw new TypeError('Transport 에는 now() 를 가진 clock 이 필요합니다 (브라우저에서는 AudioContex
t)');
    }
    this.clock = options.clock;
    this.onStep = typeof options.onStep === 'function' ? options.onStep : function () {};
    this.baseBpm = options.baseBpm != null ? options.baseBpm : 120;
    this.changes = options.changes != null ? options.changes : [];
    this.stepTicks = options.stepTicks != null ? options.stepTicks : PPQ / 4; // 기본 1/16
    this.lookahead = options.lookahead != null ? options.lookahead : DEFAULT_LOOKAHEAD_SECONDS;
    this.intervalMs = options.intervalMs != null ? options.intervalMs : DEFAULT_INTERVAL_MS;
    this.startDelay = options.startDelay != null ? options.startDelay : DEFAULT_START_DELAY_SECONDS;
    this.setTimer = options.setTimer != null ? options.setTimer : function (fn, ms) { return setInterval
(fn, ms); };
    this.clearTimer = options.clearTimer != null ? options.clearTimer : function (id) { clearInterval(i
d); };
```

```
    this.playing = false;
    this.timerId = null;
    this.cursorTick = 0;
    this.cursorTime = 0;
    this.originTick = 0;
    this.originTime = 0;
  }

  start(fromTick) {
    this.stop();
    this.playing = true;
    this.cursorTick = fromTick != null ? fromTick : 0;
    this.originTick = this.cursorTick;
    this.cursorTime = this.clock.now() + this.startDelay;
    this.originTime = this.cursorTime;
    this.pump();
    const self = this;
    this.timerId = this.setTimer(function () { self.pump(); }, this.intervalMs);
    return this;
  }

  stop() {
    if (this.timerId != null) {
      this.clearTimer(this.timerId);
      this.timerId = null;
    }
    this.playing = false;
    return this;
  }

  pump() {
    const untilTime = this.clock.now() + this.lookahead;
    const plan = planSchedule(this.cursorTick, this.cursorTime, untilTime, this.stepTicks, this.baseBpm,
this.changes);
    for (let i = 0; i < plan.events.length; i++) {
      this.onStep(plan.events[i].tick, plan.events[i].time);
    }
    this.cursorTick = plan.nextTick;
    this.cursorTime = plan.nextTime;
    return plan;
  }

  positionTick() {
    const originSeconds = ticksToSecondsWithChanges(this.originTick, this.baseBpm, this.changes);
    const nowSeconds = originSeconds + (this.clock.now() - this.originTime);
    return secondsToTicks(nowSeconds, this.baseBpm, this.changes);
  }
}
```

최신 문법으로 쓰면 (공부용)

```
const self = this;
this.timerId = this.setTimer(function () { self.pump(); }, this.intervalMs);
// 위 두 줄은 화살표 함수로:
this.timerId = this.setTimer(() => this.pump(), this.intervalMs);
```

최신 문법	실제 코드	뜻
화살표 함수 `() => this.pump()`	`const self = this; function () { self.pump(); }`	화살표 함수는 `this` 가 만들어질 때의 것을 그대로 씁니다. 실제 코드는 `self` 에 담아 씁니다
`options.clock ?? ...`	`options.clock != null ? … : …`	`null`·`undefined` 일 때 기본값을 씁니다
`class`	(같음)	실제 코드도 `class` 를 씁니다

3. 왜 이렇게

선택	이유
`clock` 을 주입받음	브라우저에서는 `AudioContext`(그 `currentTime`)를 그대로 넘기고, 시험에서는 가짜 시계를 넘겨 소리 없이 확인합니다. 나중에 오프라인 렌더(`OfflineAudioContext`, S1-11)도 같은 방식으로 씁니다
`setTimer`·`clearTimer` 도 주입	Node 시험에서 진짜 타이머를 기다리지 않고 손으로 `pump()` 를 불러 1초를 한 번에 시뮬레이션합니다
기본 타이머를 함수로 감쌈	`this.setTimer(...)` 로 전역 `setInterval` 을 그대로 부르면 `this` 가 `window` 가 아니게 되어 브라우저에서 "Illegal invocation" 이 납니다. `function (fn, ms) { return setInterval(fn, ms); }` 로 감싸 `this` 와 무관하게 부릅니다
`start` 에서 곧바로 한 번 `pump()`	첫 소리도 lookahead 안에 미리 걸려 늦지 않습니다. 타이머 첫 회(25ms 뒤)만 기다리면 시작 직후가 빕니다
`startDelay`	커서를 `now` 가 아니라 `now + 0.05` 에서 시작해, 이미 지난 시각에 예약하는 일을 막습니다
`start` 가 먼저 `stop()`	두 번 눌러도 타이머가 겹치지 않습니다
`positionTick` 은 기준점 계산	커서가 아니라 "재생 시작점(originTick, originTime)"에서 흐른 시간을 105·106으로 되돌립니다. 예약을 미리 해 둬도 화면 재생선은 실제 시각과 맞습니다
이번엔 재생·정지·위치만	pause/resume, seek, 재생 중 템포 변경은 예약 취소·재계산이 필요해 다음 함수로 미룹니다. `stop` 은 타이머만 끊으므로 이미 예약된 창(최대 0.1초)의 소리는 울린 뒤 멈춥니다

4. 시험 (Node에서 직접 실행, 23개 통과 · 가짜 시계·가짜 타이머)

확인	결과
기본값(lookahead 0.1, interval 25ms, startDelay 0.05)	✓
`clock` 없으면 `TypeError`	✓
`start` 하면 곧바로 한 번 예약, 첫 소리가 startDelay(0.05초)에, 타이머 등록	✓
25ms 씩 1초 시뮬레이션: tick이 240씩 이어지고 소리 간격이 0.125초, 늦게 예약한 소리 없음	✓
`stop` 하면 타이머 제거, 더 이상 예약 안 함	✓
`start(3840)` 위치 기준으로 시작	✓
`positionTick`: 시작 시점·+0.125초(240tick)·+0.5초(960tick)	✓
템포 변경 곡에서 `positionTick`(60 BPM 0.5초 = 480tick)	✓
`start` 두 번 = 앞 타이머 정리(겹치지 않음)	✓

JS 109 — `Transport` 위치 제어 (`pause` · `resume` · `seek`)

항목	내용
파일	`studionx/frontend/static/js/audio/scheduler.js` (같은 클래스에 메서드 추가)
작업	S1-05 오디오 엔진 기초: 여섯 번째 함수 (FR-201 재생/정지/일시정지/위치 이동)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-05-audio-engine`
다음	재생 중 템포 변경(`setTempo`), 그다음 메트로놈, 루프

1. 하는 일

108의 `Transport` 에 일시정지·이어서·위치 이동을 붙였습니다. 겸해서 `start`·`stop`·`positionTick` 이 쓰던 공통 부분을 내부 메서드(`_clearTimer`·`_anchor`·`_alignUp`·`_scheduleTimer`)로 묶었습니다(108 코드를 이렇게 다듬었습니다).

메서드	하는 일
`pause()`	지금 위치(`positionTick`)를 기억하고 타이머를 끕니다. 이후 `positionTick` 은 그 위치에 멈춥니다
`resume()`	멈춘 위치를 다음 단계 경계로 올려(최대 한 단계) 그곳부터 다시 예약합니다
`seek(tick)`	그 위치로 옮깁니다. 재생 중이면 그 위치에서 다시 예약하고, 멈춰 있으면 위치만 옮깁니다

`resume` 이 경계로 올리는 이유는, 멈추기 직전에 이미 예약된 단계와 겹치지 않게 하기 위해서입니다.

2. 코드

```
  pause() {
    if (!this.playing) return this;
    const tick = this.positionTick();
    this._clearTimer();
    this.playing = false;
    this._anchor(tick, this.clock.now());
    return this;
  }

  resume() {
    if (this.playing) return this;
    const from = this._alignUp(this.cursorTick);
    this.playing = true;
    this._anchor(from, this.clock.now() + this.startDelay);
    this.pump();
    this._scheduleTimer();
    return this;
  }

  seek(tick) {
    const wasPlaying = this.playing;
    this._clearTimer();
```

```
    this._anchor(tick, wasPlaying ? this.clock.now() + this.startDelay : this.clock.now());
    if (wasPlaying) {
      this.pump();
      this._scheduleTimer();
    }
    return this;
  }

  positionTick() {
    if (!this.playing) return this.originTick;
    const originSeconds = ticksToSecondsWithChanges(this.originTick, this.baseBpm, this.changes);
    const nowSeconds = originSeconds + (this.clock.now() - this.originTime);
    return secondsToTicks(nowSeconds, this.baseBpm, this.changes);
  }

  _clearTimer() {
    if (this.timerId != null) { this.clearTimer(this.timerId); this.timerId = null; }
  }
  _anchor(tick, time) {
    this.originTick = tick; this.originTime = time;
    this.cursorTick = tick; this.cursorTime = time;
  }
  _alignUp(tick) { return Math.ceil(tick / this.stepTicks) * this.stepTicks; }
  _scheduleTimer() {
    const self = this;
    this.timerId = this.setTimer(function () { self.pump(); }, this.intervalMs);
  }
```

최신 문법으로 쓰면 (공부용)

```
  positionTick() {
    if (!this.playing) return this.originTick;
    const originSeconds = ticksToSecondsWithChanges(this.originTick, this.baseBpm, this.changes);
    return secondsToTicks(originSeconds + this.clock.now() - this.originTime, this.baseBpm, this.change
s);
  }
```

최신 문법	실제 코드	뜻
비공개 필드 `#timerId`	`_timerId`(밑줄 이름)	진짜 비공개는 `#` 이지만, 전통 문법을 지키려고 밑줄 이름을 씁니다
`Math.ceil`	(같음)	실제 코드도 `Math.ceil` 을 씁니다

3. 왜 이렇게

선택	이유
`pause` 는 위치를 기억하고 멈춤	`positionTick` 이 `!playing` 일 때 `originTick` 을 돌려주므로, 시계가 흘러도 일시정지 중에는 재생선이 그 자리에 멈춥니다
`resume` 은 다음 단계 경계로 올림	멈추기 직전에 예약된 단계와 겹쳐 소리가 두 번 나는 것을 줄입니다(최대 한 단계 = 1/16박)
`seek` 는 재생 중일 때만 다시 예약	멈춰 있을 때는 위치만 옮기고 다음 `start`·`resume` 이 그 위치를 씁니다

선택	이유
이미 잡힌 소리 창은 남음	`stop`·`pause` 는 타이머만 끊습니다. 최대 lookahead(0.1초) 안에서 이미 예약된 소리는 울린 뒤 멈춥니다. 곧바로 끊으려면 소리를 잇는 함수에서 그 노드를 취소해야 합니다(문서에 남김)
`seek`·재생 시작에 `startDelay` 를 줌	지난 시각에 예약하는 것을 막습니다. 그래서 위치가 `originTick` 에 정확히 닿는 시각은 `startDelay` 뒤입니다(재생선이 아주 조금 앞서 보일 수 있음)
공통 부분을 내부 메서드로 묶음	`start`·`stop`·`pause`·`resume`·`seek` 이 모두 타이머 정리와 기준점 맞추기를 써서, 한 곳에 모아 실수를 줄였습니다

4. 시험 (Node에서 직접 실행, 21개 통과 · 가짜 시계·가짜 타이머)

확인	결과
일시정지: `playing` 거짓, 타이머 꺼짐, 시계가 흘러도 위치 고정	✓
이어서: 타이머 켜짐, 단계 경계에 맞춰 예약, 이후 tick이 240씩 이어짐	✓
`resume`·`pause` 두 번 = 아무 일 없음 / 타이머 한 번만 정리	✓
`seek` (멈춤): 위치만 이동, 예약 안 함	✓
`seek` (재생 중): 새 위치에서 예약, 이후 240씩 이어짐	✓
`stop` 후 멈춰 있고 위치 유지	✓
`start` 다시 = 처음 위치부터 낮은 tick 으로 예약	✓

JS 110 — `Transport.setTempo()` (재생 중 템포 변경)

항목	내용
파일	`studionx/frontend/static/js/audio/scheduler.js` (같은 클래스에 메서드 추가)
작업	S1-05 오디오 엔진 기초: 일곱 번째 함수 (FR-202 시작 BPM, FR-202a 곡 중간 템포 변경)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-05-audio-engine`
다음	메트로놈, 루프

1. 하는 일

재생을 멈추지 않고 BPM과 곡 중간 템포 변경점(`tempo_changes`)을 바꿉니다. 재생 중이면 **지금 위치를 기억하고 다음 단계 경계로 올려 그곳부터 새 템포로 다시 예약**합니다.

호출	결과
재생 중 `setTempo(60, [])` (120→60)	이후 단계 간격이 0.125초 → 0.25초
같은 자리에서 `setTempo(120, [{tick:3840,bpm:240}])`	3840 뒤 간격이 0.125초 → 0.0625초
멈춰 있을 때 `setTempo`	위치 그대로, 예약 안 함, 다음 재생에 반영
일시정지 중 `setTempo`	위치 고정 유지

2. 코드

```
  setTempo(baseBpm, changes) {
    if (typeof baseBpm !== 'number' || !isFinite(baseBpm) || baseBpm <= 0) throw new RangeError('baseBpm
은 0 보다 큰 숫자여야 합니다: ' + baseBpm);
    const tick = this.positionTick();
    this.baseBpm = baseBpm;
    this.changes = changes != null ? changes : [];
    if (this.playing) {
      const from = this._alignUp(tick);
      this._clearTimer();
      this._anchor(from, this.clock.now() + this.startDelay);
      this.pump();
      this._scheduleTimer();
    }
    return this;
  }
```

최신 문법으로 쓰면 (공부용)

```
  setTempo(baseBpm, changes = []) {
    if (!Number.isFinite(baseBpm) || baseBpm <= 0) throw new RangeError(`baseBpm 은 0 보다 큰 숫자여야 합
니다: ${baseBpm}`);
    // …나머지 같음
  }
```

최신 문법	실제 코드	뜻
기본 인자 `changes = []`	`changes != null ? changes : []`	값이 없으면 빈 배열을 씁니다
`Number.isFinite(x)`	`typeof x !== 'number' || !isFinite(x)`	숫자이면서 NaN·Infinity가 아닌지 한 번에 봅니다
템플릿 문자열	`'…' + x`	글자와 값을 잇습니다

3. 왜 이렇게

선택	이유
위치를 **바꾸기 전에** 읽음	`positionTick()` 은 지금 `baseBpm`·`changes` 로 계산합니다. 먼저 위치를 구해 두고 템포를 바꿔야 옛 템포 기준 위치가 나옵니다
재생 중이면 단계 경계로 올림	`resume` 과 같은 방식입니다. 커서가 이미 새 단계 경계(1/16박)에 맞춰져 있어, 이후 단계도 격자에 맞습니다. 대신 재생선이 최대 한 단계 앞으로 살짝 뛸 수 있습니다
커서를 다시 잡고 `pump()`	새 템포로 다시 계산해야 하므로, 옛 커서의 미래 시각은 버리고 지금 위치부터 새로 잡습니다
이미 예약된 창은 옛 템포로 울림	커서 이후(최대 lookahead 0.1초)는 이미 예약되어 취소할 수 없습니다. 그 뒤부터 새 템포가 적용됩니다 (문서에 남김)
멈춰 있을 때는 다시 잡지 않음	`positionTick` 이 `originTick` 을 돌려주므로 위치가 그대로입니다. 다음 `start`·`resume` 이 새 템포를 씁니다
`baseBpm` 을 먼저 검사	멈춰 있을 때는 `pump()` 를 안 불러 105의 검사가 안 돌 수 있어 여기서 직접 검사합니다

4. 시험 (Node에서 직접 실행, 18개 통과 · 가짜 시계·가짜 타이머)

확인	결과
재생 중 120→60: 이후 tick이 240씩, 시간 간격이 0.25초	✓
변경 전 간격이 0.125초였는지	✓
변경 위치가 단계 경계에 맞고 크게 안 뛰는지	✓
멈춰 있을 때: 예약 안 함, 위치 유지, 필드만 갱신	✓
일시정지 중: 위치 고정 유지, `playing` 그대로	✓
재생 중 템포 변경점 추가(3840 뒤 간격 0.0625초)	✓
`baseBpm` 0·NaN 이면 `RangeError`	✓

JS 111 — `metronomeClick()` (메트로놈 박·강박 판정)

항목	내용
파일	`studionx/frontend/static/js/audio/metronome.js` (새 파일, 소리를 쓰지 않는 순수 계산)
작업	S1-05 오디오 엔진 기초: 여덟 번째 함수 (FR-203 메트로놈)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-05-audio-engine`
다음	메트로놈 클래스(on/off·클릭 소리·카운트인 연결), 그다음 루프

1. 하는 일

어떤 tick이 메트로놈 박에 놓이는지, 그중 마디 첫 박(강박)인지 판정합니다. 스케줄러(`Transport`)가 `onStep(tick, time)`으로 넘겨준 단계마다 이 함수를 불러, `null`이 아니면 클릭 소리를 냅니다.

박 길이는 **박자 분모(den)** 로 정합니다. 4/4는 4분음표(960 tick), 6/8은 8분음표(480 tick)입니다.

입력 (tick, 분자, 분모)	결과
0, 4, 4	`{beatIndex:0, accent:true}` (강박)
960, 4, 4	`{beatIndex:1, accent:false}`
240, 4, 4	`null` (박 사이)
480, 6, 8	`{beatIndex:1, accent:false}`
−3840, 4, 4	`{beatIndex:0, accent:true}` (카운트인 마디 첫 박)

2. 코드

```
import { PPQ } from './timing.js';

export const TIME_SIG_DEN_VALUES = [1, 2, 4, 8, 16, 32];

export function metronomeClick(tick, timeSigNum, timeSigDen) {
  if (typeof tick !== 'number' || !isFinite(tick)) throw new RangeError('tick 은 유한한 숫자여야 합니다:
' + tick);
  if (!Number.isInteger(timeSigNum) || timeSigNum < 1 || timeSigNum > 32) throw new RangeError('timeSigN
um 은 1~32 정수여야 합니다: ' + timeSigNum);
  if (TIME_SIG_DEN_VALUES.indexOf(timeSigDen) === -1) throw new RangeError('timeSigDen 은 1,2,4,8,16,32
중 하나여야 합니다: ' + timeSigDen);

  const beatTicks = (PPQ * 4) / timeSigDen;
  const measureTicks = beatTicks * timeSigNum;

  let offset = tick % measureTicks;
  if (offset < 0) offset += measureTicks;
  if (Math.abs(offset % beatTicks) > 1e-9) return null;

  const beatIndex = Math.round(offset / beatTicks);
```

```
  return { beatIndex: beatIndex, accent: beatIndex === 0 };
}
```

최신 문법으로 쓰면 (공부용)

```
export const metronomeClick = (tick, timeSigNum, timeSigDen) => {
  const beatTicks = (PPQ * 4) / timeSigDen;
  const measureTicks = beatTicks * timeSigNum;
  const offset = ((tick % measureTicks) + measureTicks) % measureTicks;
  if (offset % beatTicks !== 0) return null;
  const beatIndex = Math.round(offset / beatTicks);
  return { beatIndex, accent: beatIndex === 0 };
};
```

최신 문법	실제 코드	뜻
`((x % m) + m) % m`	`let offset = …; if (offset < 0) offset += m;`	음수도 0 이상으로 만드는 나머지입니다
`{ beatIndex, accent }` (속기)	`{ beatIndex: beatIndex, accent: … }`	이름이 같은 변수를 그대로 담습니다
`TIME_SIG_DEN_VALUES.includes(x)`	`indexOf(x) === -1`	배열에 값이 있는지 봅니다(전통 문법)

3. 왜 이렇게

선택	이유
소리와 분리한 순수 함수	클릭 소리를 내지 않고도 박·강박 판정을 Node로 시험합니다. 소리는 다음 함수에서 `AudioContext` 노드로 냅니다
박 길이를 분모로 정함	6/8은 8분음표가 박이라 분모 8이 자연스럽습니다. 강박(마디 첫 박)이 6/8에서도 첫 8분음표에 옵니다
음수 tick도 마디 격자로	카운트인(FR-203, 1마디)은 0보다 앞에서 울립니다. −마디길이도 강박이 되어 카운트인이 한 마디로 들립니다
`1e-9` 허용 오차	`%` 계산에 아주 작은 소수 오차가 날 수 있어, 박 위인지 확인할 때 아주 작은 여유를 둡니다. 소수 tick(격자 아님)은 `null`
`Number.isInteger` 로 분자 검사	분자는 정수만 뜻이 있습니다(1~32). `DB.md` 의 CHECK 와 같은 범위입니다
`round` 로 박 번호	부동소수 오차가 있어도 0~분자−1 의 정수 박 번호가 나옵니다
단계가 박보다 촘촘해야 함	스케줄러 단계(`stepTicks`)가 박 길이보다 크면 일부 박을 건너뜁니다. 기본 1/16(240)은 분모 16까지(240 tick) 덮습니다

4. 시험 (Node에서 직접 실행, 35개 통과)

확인	결과
4/4 강박·1~3박·다음 마디, 박 사이 null	✓
3/4, 6/8, 7/8, 2/2, 1/4(매 박 강박)	✓
음수 tick(카운트인 −3840 강박, −960=3박, −240 null)	✓
소수 tick(100.5)은 null	✓
분자·분모 오류(NaN·0·33·1.5·분모 3·0) 시 `RangeError`	✓

확인	결과
`TIME_SIG_DEN_VALUES` 가 [1,2,4,8,16,32]	✓

JS 112 — `Metronome` 클래스 · `countInTicks()` (메트로놈 on/off·카운트인)

항목	내용
파일	`studionx/frontend/static/js/audio/metronome.js` (111에 이어 추가)
작업	S1-05 오디오 엔진 기초: 아홉 번째 함수 (FR-203 메트로놈 on/off·카운트인)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-05-audio-engine`
다음	루프(S1-05 마지막)

1. 하는 일

111의 순수 판정(`metronomeClick`) 위에 on/off와 클릭 소리 연결을 얹었습니다. 소리는 페이지가 넘긴 `click(time, accent)` 콜백에서 냅니다(AudioContext는 페이지 담당). 겸해서 카운트인 길이를 돌려주는 `countInTicks()`를 더했습니다.

대상	하는 일
`new Metronome({click, enabled?, timeSigNum?, timeSigDen?})`	`click` 콜백 필수(없으면 `TypeError`). 기본은 꺼짐·4/4
`setEnabled(on)`	메트로놈 켜기/끄기 (FR-203)
`setTimeSignature(num, den)`	박자 바꾸기 (FR-202a)
`step(tick, time)`	켜져 있고 박 위이면 `click(time, accent)`를 부르고 `{beatIndex, accent}` 반환, 아니면 `null`
`countInTicks(num, den)`	한 마디 길이(tick). 트랜스포트를 그만큼 앞에서 시작하면 카운트인이 들림

페이지 사용 예(개념):

```
const metronome = new Metronome({ click: playClick, enabled: true, timeSigNum: 4, timeSigDen: 4 });
transport.onStep = function (tick, time) { metronome.step(tick, time); };
transport.start(-countInTicks(4, 4));  // 한 마디 카운트인 뒤 0 에서 곡 시작
```

2. 코드

```
function assertTimeSig(timeSigNum, timeSigDen) {
  if (!Number.isInteger(timeSigNum) || timeSigNum < 1 || timeSigNum > 32) throw new RangeError('timeSigNum 은 1~32 정수여야 합니다: ' + timeSigNum);
  if (TIME_SIG_DEN_VALUES.indexOf(timeSigDen) === -1) throw new RangeError('timeSigDen 은 1,2,4,8,16,32 중 하나여야 합니다: ' + timeSigDen);
}

export function countInTicks(timeSigNum, timeSigDen) {
  assertTimeSig(timeSigNum, timeSigDen);
  return ((PPQ * 4) / timeSigDen) * timeSigNum;
```

```
}

export class Metronome {
  constructor(options) {
    options = options || {};
    if (typeof options.click !== 'function') throw new TypeError('click 콜백(함수)이 필요합니다');
    this.click = options.click;
    this.enabled = options.enabled != null ? !!options.enabled : false;
    this.timeSigNum = options.timeSigNum != null ? options.timeSigNum : 4;
    this.timeSigDen = options.timeSigDen != null ? options.timeSigDen : 4;
    assertTimeSig(this.timeSigNum, this.timeSigDen);
  }
  setEnabled(on) { this.enabled = !!on; return this; }
  setTimeSignature(timeSigNum, timeSigDen) { assertTimeSig(timeSigNum, timeSigDen); this.timeSigNum = ti
meSigNum; this.timeSigDen = timeSigDen; return this; }
  step(tick, time) {
    if (!this.enabled) return null;
    const info = metronomeClick(tick, this.timeSigNum, this.timeSigDen);
    if (info === null) return null;
    this.click(time, info.accent);
    return info;
  }
}
```

최신 문법으로 쓰면 (공부용)

```
  constructor({ click, enabled = false, timeSigNum = 4, timeSigDen = 4 } = {}) {
    if (typeof click !== 'function') throw new TypeError('click 콜백(함수)이 필요합니다');
    // …
  }
  setEnabled(on) { this.enabled = Boolean(on); return this; }
```

최신 문법	실제 코드	뜻
구조 분해 기본값 `{ click, enabled = false } = {}`	`options = options	
`Boolean(on)`	`!!on`	값을 참/거짓으로 바꿉니다
선택적 체이닝 `options?.click`	`options.click`	앞이 없을 때만 안전하게 접근합니다(전통 문법에서는 `options || {}` 로 먼저 채움)

3. 왜 이렇게

선택	이유
소리는 콜백으로 주입	클릭 소리(AudioContext)를 파일 밖으로 빼서 Node로 on/off·박 판정을 시험합니다. 소리 내는 부분은 페이지가 담당
`assertTimeSig` 를 따로 둠	111·`countInTicks`·클래스 세 곳이 같은 검사를 씁니다. 한 곳에 모아 규칙이 어긋나지 않게 함
기본 꺼짐	메트로놈은 사용자가 켜야 들립니다(FR-203 on/off). 처음부터 소리가 나면 놀람
꺼져 있으면 바로 `null`	판정 전에 돌아가 불필요한 계산을 줄입니다
`step` 이 판정 결과를 돌려줌	페이지나 시험에서 "이번 단계에 울렸는지" 확인할 수 있습니다
카운트인은 트랜스포트 시작점	카운트인은 0보다 앞에서 울리는 것이라, 클래스가 소리를 따로 세지 않고 시작 위치로 처리합니다. 음수 tick은 111이 이미 격자로 다룸

선택	이유
`setEnabled`·`setTimeSignature` 가 self 반환	잇달아 부르기 쉽게 합니다(`m.setEnabled(true).setTimeSignature(6,8)`). 110 `setTempo` 와 같은 방식

4. 시험 (Node에서 직접 실행, 31개 통과 · 클릭 콜백 기록)

확인	결과
`countInTicks` 4/4=3840, 6/8·3/4=2880, 7/8=3360, 오류 시 `RangeError`	✓
`click` 콜백 없으면 `TypeError`	✓
기본 꺼짐·4/4, 꺼진 채 `step` 은 `null`·클릭 없음	✓
켜면 강박·박에서 클릭, `time`·`accent` 전달, 박 사이는 `null`	✓
끄면 다시 안 울림, `setEnabled` self 반환	✓
`setTimeSignature(6,8)` 로 박 격자 변경(8분음표 480), self 반환, 오류 시 `RangeError`	✓
생성 시 박자 지정(3/4) 반영	✓
111 `metronomeClick` 회귀	✓

JS 113 — 루프 (`Transport.setLoop()` · `planSchedule` 루프)

항목	내용
파일	`studionx/frontend/static/js/audio/scheduler.js` (`planSchedule` 인자 추가 + `Transport.setLoop`)
작업	S1-05 오디오 엔진 기초: 열 번째 함수 (FR-204 루프)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-05-audio-engine`
다음	S1-05 마무리(전체 회귀·커밋 결정). 이후 S1-06

1. 하는 일

재생 위치가 루프 끝(tick)에 닿으면 시작(tick)으로 되돌아갑니다. 시간(초)은 계속 흐르고 tick만 되돌아갑니다.

대상	하는 일
`Transport.setLoop(start, end, enabled)`	루프 구간·on/off 설정. 재생 중이면 지금 위치로 다시 예약
`planSchedule(..., loop)`	`{start, end}`를 받으면 끝을 지나는 단계의 tick을 시작으로 되돌림
`Transport.positionTick()`	루프가 켜져 있고 끝을 지나면 시작 기준으로 되돌린 tick을 돌려줌

```
transport.setLoop(0, 3840, true);   // 0~3840 tick(1마디) 구간 반복
transport.start(0);
```

2. 코드

`planSchedule`에 `loop` 인자를 더했습니다(없으면 지금까지와 같음).

```
  if (loop != null && (!(loop.end > loop.start))) throw new RangeError('loop 은 start < end 여야 합니다:
' + loop.start + ' ~ ' + loop.end);

  const loopLen = loop != null ? loop.end - loop.start : 0;
  function wrap(t) {
    if (loop == null) return t;
    return t >= loop.end ? loop.start + ((t - loop.start) % loopLen) : t;
  }
  // 반복 안:
  //   const rawTick = tick + stepTicks;
  //   const delta = f(rawTick) - f(tick);   // f = ticksToSecondsWithChanges
  //   tick = wrap(rawTick);
  //   time += delta;
```

`Transport`에 구간 필드와 `setLoop`을 더했습니다.

```
  setLoop(startTick, endTick, enabled) {
    if (startTick != null) this.loopStart = startTick;
    if (endTick != null) this.loopEnd = endTick;
    if (enabled != null) this.loopEnabled = !!enabled;
    if (this.loopEnabled && !(this.loopEnd > this.loopStart)) throw new RangeError('loopEnd > loopStart
```

```
여야 합니다: ' + this.loopStart + ' ~ ' + this.loopEnd);
    if (this.playing) {
      const from = this._alignUp(this.positionTick());
      this._clearTimer();
      this._anchor(from, this.clock.now() + this.startDelay);
      this.pump();
      this._scheduleTimer();
    }
    return this;
  }
  _loopRegion() {
    if (this.loopEnabled && this.loopEnd > this.loopStart) return { start: this.loopStart, end: this.loo
pEnd };
    return null;
  }
  // positionTick: 구한 tick 이 끝을 지나면 시작 기준으로 되돌림
  //   const loop = this._loopRegion();
  //   if (loop !== null && tick >= loop.end) return loop.start + ((tick - loop.start) % (loop.end - loo
p.start));
```

최신 문법으로 쓰면 (공부용)

```
const wrap = t => (loop == null || t < loop.end) ? t : loop.start + ((t - loop.start) % (loop.end - loo
p.start));
setLoop(startTick, endTick, enabled) { /* … */ return this; }
```

최신 문법	실제 코드	뜻
기본 매개변수 · 화살표 함수	`function wrap(t) {…}`	작은 도우미 함수를 안에서 정의합니다
`??=`	`if (enabled != null) this.loopEnabled = !!enabled;`	값이 있을 때만 바꿉니다(전통 문법)
`String.prototype`·템플릿	`'…' + a + ' ~ ' + b`	글자와 값을 잇습니다

3. 왜 이렇게

선택	이유
되돌림을 tick 공간의 나머지로	소리 시간(초)은 계속 커지고 tick만 접힙니다. 오디오 시각은 한 방향으로 흘러야 하기 때문입니다
끝을 지나는 단계의 걸린 시간은 "끝 직전" 기준	`delta = f(tick+step) - f(tick)` 로 계산해, 되돌아간 뒤에도 그 단계가 자연스럽게 이어집니다. 그래서 루프를 한 바퀴 도는 시간이 곡 템포와 맞습니다
`planSchedule` 이 계산을 담당	루프는 예약 목록을 만드는 일이라 순수 함수에 넣어야 Node로 시험할 수 있습니다
`Transport` 는 구간만 넘김	`_loopRegion()` 이 "켜져 있고 길이가 있으면" 구간을 돌려줘, 꺼졌거나 잘못된 값이면 `null` 이 되어 지금까지와 같게 돕니다
재생 중 `setLoop` 은 다시 예약	커서가 옛 구간 기준으로 앞서 있을 수 있어, 지금 위치로 커서를 되돌려 새 구간을 반영합니다 (`setTempo` 와 같은 방식, 최대 한 단계 앞으로 살짝 뛺)
멈춰 있을 때는 예약 안 함	위치만 바꾸고, 다음 `start`·`resume` 이 새 구간을 씁니다
아직 잡힌 창은 옛 구간대로	재생 중 설정하면 최대 lookahead(0.1초) 안의 이미 예약된 소리는 그대로 울립니다(문서에 남김)

선택	이유
시작이 구간보다 뒤여도 동작	`wrap` 은 `t >= loop.end` 일 때만 접으므로, 재생 위치가 구간에 들어와 끝을 지나면 반복이 시작됩니다. 템포 변경점이 구간에 걸쳐 있으면 오차가 날 수 있음(프로토타입 한계)

4. 시험 (Node에서 직접 실행, 27개 통과 · 가짜 시계·가짜 타이머)

확인	결과
`planSchedule` 루프: 16단계(2초)마다 tick 0으로 되돌아감, 시간 간격은 0.125초로 연속	✓
구간 안쪽 시작(`loop {960,3840}`)에서 시작해도 960으로 되돌아감, 모든 tick이 구간 안	✓
`nextTick`·`nextTime` 커서가 루프를 반영해 이어짐	✓
루프 없으면 지금까지와 같음(선형)	✓
루프 값 오류(start=end, start>end) 시 `RangeError`	✓
재생 중 tick 720 다음이 0으로 되돌아감, 시간은 계속 증가, 위치가 구간 안	✓
루프를 끄면 선형으로 이어짐	✓
멈춰 있을 때 `setLoop` 은 예약 안 함, 필드만 설정	✓
재생 중 루프 켜면 이후 예약이 구간을 따름, 위치가 구간 안	✓
`setLoop` self 반환, 생성 시 잘못된 루프는 `RangeError`	✓
회귀: 107(26)·108(23)·109(21)·110(18) 모두 통과	✓

JS 114 — `AudioEngine` · `createClickPlayer()` (AudioContext 연결)

항목	내용
파일	`studionx/frontend/static/js/audio/engine.js` (새 파일)
작업	S1-05 오디오 엔진 기초: 열한 번째 함수 (FR-201~205 AudioContext 연결)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-05-audio-engine`
다음	데모 페이지(재생선·메트로놈 소리) 브라우저 확인, 그 뒤 S1-05 종료

1. 하는 일

104~113의 순수 계산과 `Transport`·`Metronome`을 **실제 `AudioContext`에 연결**합니다. `clock`은 `AudioContext.currentTime`, 타이머는 `setInterval`, 클릭 소리는 오실레이터로 냅니다.

대상	하는 일
`createClickPlayer(context, destination)`	짧은 사각파로 "딱" 소리를 내는 함수를 만듦. 강박은 높고 크게 (1600Hz)
`new AudioEngine({context, onStep, baseBpm, timeSigNum, ...})`	컨텍스트·트랜스포트·메트로놈을 묶음. `context`를 안 넘기면 `window.AudioContext` 사용
`start({fromTick, countIn})`	(브라우저) 컨텍스트를 깨우고 재생. `countIn`이면 한 마디 카운트인 뒤 0에서 시작
`stop() · pause() · resume() · seek(tick)`	재생 제어
`setTempo · setLoop · setMetronome · setTimeSignature`	트랜스포트·메트로놈에 전달
`positionTick()`	화면 재생선용 위치(tick)

페이지 사용 예(개념):

```
const engine = new AudioEngine({ metronomeEnabled: true, onStep: function (tick, time) { /* 악기 소리 */
} });
engine.start({ countIn: true });
requestAnimationFrame(function draw() { var t = engine.positionTick(); /* 재생선 그리기 */ requestAnimati
onFrame(draw); });
```

2. 코드

```
export function createClickPlayer(context, destination) {
  if (destination == null) destination = context.destination;
  return function playClick(time, accent) {
    const osc = context.createOscillator();
    const gain = context.createGain();
    osc.type = 'square';
    osc.frequency.value = accent ? 1600 : 1000;
    const dur = 0.03;
    gain.gain.setValueAtTime(accent ? 0.3 : 0.15, time);
```

```
    gain.gain.exponentialRampToValueAtTime(0.0001, time + dur);
    osc.connect(gain);
    gain.connect(destination);
    osc.start(time);
    osc.stop(time + dur + 0.01);
  };
}

export class AudioEngine {
  constructor(options) {
    options = options || {};
    let context = options.context != null ? options.context : null;
    if (context == null) {
      const Ctor = (typeof window !== 'undefined') ? (window.AudioContext || window.webkitAudioContext)
: null;
      if (Ctor == null) throw new Error('AudioContext 를 만들 수 없습니다 — options.context 를 넘겨 주세
요');
      context = new Ctor();
    }
    this.context = context;
    this.destination = options.destination != null ? options.destination : context.destination;
    const self = this;
    this.clock = { now: function () { return self.context.currentTime; } };
    this.onStep = typeof options.onStep === 'function' ? options.onStep : function () {};
    this.metronome = new Metronome({ click: createClickPlayer(context, this.destination), enabled: …, ti
meSigNum: …, timeSigDen: … });
    this.transport = new Transport({ clock: this.clock, …, onStep: function (tick, time) { self._onStep
(tick, time); } });
  }
  _onStep(tick, time) { this.metronome.step(tick, time); this.onStep(tick, time); }
  start(options) {
    options = options || {};
    this._resumeContext();
    let from = options.fromTick != null ? options.fromTick : 0;
    if (options.countIn) from = -countInTicks(this.metronome.timeSigNum, this.metronome.timeSigDen);
    this.transport.start(from);
    return this;
  }
  _resumeContext() {
    const ctx = this.context;
    if (ctx != null && ctx.state === 'suspended' && typeof ctx.resume === 'function') { try { ctx.resume
(); } catch (e) {} }
  }
}
```

최신 문법으로 쓰면 (공부용)

```
const Ctor = globalThis.AudioContext ?? globalThis.webkitAudioContext;
const from = options.fromTick ?? 0;
ctx.resume?.().catch(() => {});
```

최신 문법	실제 코드	뜻
`??` (널 병합)	`options.fromTick != null ? options.fromTick : 0`	null/undefined일 때만 기본값
선택적 호출 `resume?.()`	`typeof ctx.resume === 'function'` 확인	함수가 있을 때만 호출

최신 문법	실제 코드	뜻
`globalThis`	`typeof window !== 'undefined'`	브라우저 전역을 안전하게 봄

3. 왜 이렇게

선택	이유
얇은 층으로 묶음	순수 계산(104~113)은 그대로 두고, 실제 소리·시계만 여기서 얹습니다. 계산 로직을 다시 시험할 필요가 없습니다
`context` 주입 가능	Node에서 가짜 컨텍스트로 배선(예약·클릭·resume)까지 시험합니다. 실제 소리는 브라우저에서 확인
`clock`을 `context.currentTime`으로	오디오 시계는 오실레이터 예약과 같은 시계라 소리가 어긋나지 않습니다
`onStep`을 트랜스포트에 넘김	페이지가 악기 소리를 얹을 자리입니다. 메트로놈을 먼저 울린 뒤 페이지 `onStep`을 부릅니다
클릭 소리를 오실레이터로	샘플 파일 없이 짧은 사각파로 "딱" 소리를 냅니다. 강박은 1600Hz·큰 소리로 구분
`start`에서 `resume`	브라우저는 사용자 조작 뒤에야 소리가 나므로, 멈춰 있으면 깨웁니다. 실패해도 무시하고 진행
카운트인을 `start` 옵션으로	트랜스포트를 음수 위치에서 시작하면 111의 음수 격자가 그대로 카운트인 클릭을 만듭니다
`positionTick` 전달	재생선 그리기는 페이지가 `requestAnimationFrame`으로 이 값을 읽습니다

4. 시험 (Node에서 직접 실행, 41개 통과 · 가짜 AudioContext·가짜 타이머)

확인	결과
`createClickPlayer`: 오실레이터 생성·주파수(1600/1000)·시작/정지 시각·gain 연결	✓
`context` 없이 브라우저도 아니면 `Error`	✓
엔진 구성: 트랜스포트·메트로놈·destination·기본 꺼짐	✓
`start`: suspended → `resume` 1회, 스텝 예약(tick 0·startDelay), 다음 스텝 tick 240	✓
메트로놈 켜면 박마다 클릭, 첫 클릭 강박, 클릭 간격 0.5초(120bpm 4/4)	✓
메트로놈 끄면 클릭 없음, 그래도 스텝은 예약	✓
카운트인: 첫 스텝 −3840(한 마디 전), 강박 클릭	✓
`setTempo`·`setLoop`·`setTimeSignature` 전달, self 반환	✓
`pause`·`resume`·`seek`·`stop` 동작·self 반환	✓